1 Contest 1

2 Mathematics 1

3 Data structures 3

4 Numerical 4

5 Number theory 5

6 Combinatorial 5

7 Graph 5

8 Geometry 6

9 Strings 8

10 Various 8

Contest (1)

template.cpp

14 lines

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

Makefile

21 lines

```
TARGET = A
FLAGS = -std=c++20 -Wall -Wconversion -Wfatal-errors -g -
    fsanitize=undefined,address

all: $(TARGET)

$(TARGET): $(TARGET).cpp
  g++ $(FLAGS) $(TARGET).cpp -o $(TARGET)

run: $(TARGET)
  ./$(TARGET)

test: $(TARGET)
  @for f in $(TARGET)*.in; do \
    echo "== $$f expected:"; cat $${f%.in}.ans; \
    echo "-- got:"; ./$(TARGET) < $$f | tee out; echo; \
    diff -b out $${f%.in}.ans > /dev/null && echo "-- ok" ||
        echo "-- WRONG"; \
  done

hash: # make hash < file: hash ignoring whitespace and comments
  @cpp -dD -P -fpreprocessed | tr -d '[:space:]' \
    | md5sum | cut -c-6
```

troubleshoot.txt

52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

Mathematics (2)

2.1 Standard library

StdMath.cpp

Description: C++20, all in bits/stdc++.h. Here a, b are int/ll, u is unsigned/ull (the <bit> functions only take unsigned) and x, y, t are double. long is 32-bit on Windows (Codeforces): use long long.

0fcf02, 30 lines

```
gcd(a, b); lcm(a, b);
__lg(a);           // floor(log2(a)), exact
bit_width(u);      // floor(log2(u)) + 1
bit_ceil(u);       // smallest power of 2 >= u
has_single_bit(u); popcount(u); __builtin_popcountll(a);
countl_zero(u); countr_zero(u); rotl(u, k); rotr(u, k);
log2(x);           // inexact for big integers
atan2(y, x);       // angle of (x, y) in [-pi, pi]
hypot(x, y); sqrt(x); cbrt(x);
midpoint(a, b);    // (a + b) / 2 without overflow
lerp(x, y, t);     // x + t * (y - x)

INT_MAX    == 2147483647;            // ~2.1e9
UINT_MAX   == 4294967295;            // ~4.3e9
LLONG_MAX  == 9223372036854775807;   // ~9.2e18
ULLONG_MAX == 18446744073709551615;  // ~1.8e19
INT_MIN == -INT_MAX - 1; LLONG_MIN == -LLONG_MAX - 1;
numeric_limits<ll>::max(); numeric_limits<ll>::min();
__int128 big;                        // ~1.7e38, no cin/cout
DBL_DIG == 15; LDBL_DIG == 18;       // significant digits
DBL_MAX;                             // ~1.8e308

accumulate(all(v), 0LL);               // 0 would sum in int
partial_sum(all(v), p.begin());        // v[0] + ... + v[i]
adjacent_difference(all(v), d.begin()); // v[i] - v[i-1]
sort(all(v)); do {} while (next_permutation(all(v)));
prev_permutation(all(v));              // false on wrap-around

numbers::pi; numbers::e; numbers::phi; numbers::sqrt2;
numbers::pi_v<long double>;
```

2.2 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

2.3 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

2.4 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$\cos v + \cos w = 2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$

$$(V+W)\tan(v-w)/2 = (V-W)\tan(v+w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a\cos x + b\sin x = r\cos(x-\phi)$$
$$a\sin x + b\cos x = r\sin(x+\phi)$$

where $r = \sqrt{a^2+b^2}, \phi = \operatorname{atan2}(b, a)$.

2.5 Geometry

2.5.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a+b+c}{2}$

Area: $A = \sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

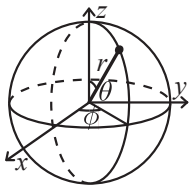
2.5.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef\cdot\sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.5.3 Spherical coordinates



$$x = r\sin\theta\cos\phi \qquad r = \sqrt{x^2+y^2+z^2}$$
$$y = r\sin\theta\sin\phi \qquad \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2})$$
$$z = r\cos\theta \qquad \phi = \operatorname{atan2}(y, x)$$

2.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$
$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$
$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$
$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$
$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$
$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Counting

Ways to pick r of n items: $\binom{n}{r} = \frac{n!}{r!(n-r)!}$ unordered, ${}^nP_r = \frac{n!}{(n-r)!}$ ordered, and $\binom{n+r-1}{r}$ unordered with repetition. Also $\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}$.

Answers are often mod $p = 10^9 + 7$ or 998 244 353 (both prime). Reduce after every $+$, $-$ and $\cdot$ using `ll`, and write `(a - b + p) % p`, as `%` can be negative. To divide by b, multiply by $b^{-1} \equiv b^{p-2}$ (Fermat).

ModBinomial.cpp

Description: $\binom{n}{r}$ and ${}^nP_r$ mod a prime, 0 if $r < 0$ or $r > n$.
Time: $\mathcal{O}(N)$ for init, $\mathcal{O}(1)$ per query

92ec98, 23 lines

```
const ll mod = 1e9 + 7; // or 998244353
ll modpow(ll b, ll e) { // b^e, and 1/b if e = mod - 2
  ll r = 1;
  for (b %= mod; e; e /= 2, b = b * b % mod)
    if (e & 1) r = r * b % mod;
  return r;
}
const int N = 1e6;
ll f[N + 1], fi[N + 1]; // n! and 1/n!
void init() { // call once
  f[0] = 1;
  rep(i,1,N+1) f[i] = f[i-1] * i % mod;
  fi[N] = modpow(f[N], mod - 2);
  for (int i = N; i > 0; i--) fi[i-1] = fi[i] * i % mod;
}
ll nCr(int n, int r) {
  if (r < 0 || r > n) return 0;
  return f[n] * fi[r] % mod * fi[n-r] % mod;
}
ll nPr(int n, int r) {
  if (r < 0 || r > n) return 0;
  return f[n] * fi[n-r] % mod;
}
```

2.8.2 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n, p)$, $n = 1, 2, \ldots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k}p^k(1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\operatorname{Bin}(n, p)$ is approximately $\operatorname{Po}(np)$ for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\operatorname{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\text{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

2.8.3 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\text{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. With transition matrix $\mathbf{P} = (p_{ij})$, $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$ (columns sum to 1), the distribution after n steps is $\mathbf{p}^{(n)} = \mathbf{P}^n\mathbf{p}^{(0)}$, where $\mathbf{p}^{(0)}$ is the initial distribution. Compute it in $O(S^3 \log n)$ for S states with `Matrix.h`: `(P^n) * p0` (the brackets are needed, as $\star$ binds tighter than `^`). With doubles the round-off grows like $n \cdot 10^{-16}$, so for big n rescale each column to sum 1 after every multiply, as `longRun` below does.

Steady state. π is stationary if $\mathbf{P}\pi = \pi$ and $\sum_i \pi_i = 1$. It is unique iff the chain has one closed class (set of states it cannot leave), e.g. if it is *irreducible* (every state can reach every state); then $\pi_i = 1/\mathbb{E}(T_i)$, where T_i is the time between two visits to state i. If it is also *aperiodic* (the gcd of cycle lengths is 1), $\mathbf{p}^{(n)} \to \pi$ from any start; if it is periodic, $\mathbf{p}^{(n)}$ keeps cycling and π is its average over time. With several closed classes, where the chain ends up depends on $\mathbf{p}^{(0)}$. `stationary` below finds a unique π exactly; `longRun` handles every case.

Diagonalising. If $\mathbf{P} = VDV^{-1}$ with $D = \text{diag}(\lambda_1, \ldots, \lambda_S)$ and eigenvectors $\mathbf{v}_i$ as the columns of V, then $\mathbf{P}^n = VD^nV^{-1}$, so writing $\mathbf{p}^{(0)} = \sum_i c_i\mathbf{v}_i$ gives $\mathbf{p}^{(n)} = \sum_i c_i\lambda_i^n\mathbf{v}_i$. $\lambda_1 = 1$ gives π; in an irreducible aperiodic chain all other $|\lambda_i| < 1$, so their terms decay like $|\lambda_2|^n$. For two states with $a = \Pr(0 \to 1)$, $b = \Pr(1 \to 0)$:

$$\pi = \tfrac{1}{a+b}(b, a), \quad \mathbf{p}^{(n)} = \pi + (1 - a - b)^n(\mathbf{p}^{(0)} - \pi).$$

SteadyState.cpp

Description: Steady state of a Markov chain, where P.d[i][j] = $\Pr(j \to i)$. stationary finds π exactly if it is unique (one closed class, e.g. irreducible; periodic is fine), and returns false otherwise. It solves $(P - I)\pi = 0$. longRun works for any chain and returns the long-run distribution from p0: the average over time if periodic, and depending on p0 if there are several closed classes. It computes $((P + I)/2)^{2^{60}}$, which is aperiodic with the same stationary distributions, and rescales each column to sum 1 against round-off.
Time: $\mathcal{O}(S^3)$

"../data-structures/Matrix.h", "../numerical/SolveLinear.h" 11374e, 23 lines

```
const int S = 3; // number of states
typedef array<double, S> V;
V longRun(Matrix<double, S> P, V p0) {
  rep(i,0,S) rep(j,0,S) P.d[i][j] = (P.d[i][j] + (i == j)) / 2;
  rep(k,0,60) {
    P = P * P;
    rep(j,0,S) {
      double t = 0;
      rep(i,0,S) t += P.d[i][j];
      rep(i,0,S) P.d[i][j] /= t;
    }
  }
  return P * p0;
}
bool stationary(Matrix<double, S> P, V& pi) {
  vector<vd> A(S, vd(S)); vd b(S), x(S);
  rep(i,0,S) rep(j,0,S) A[i][j] = P.d[i][j] - (i == j);
  rep(j,0,S) A[S-1][j] = 1; // last row: sum(pi) = 1
  b[S-1] = 1;
  if (solveLinear(A, b, x) < S) return false;
  rep(i,0,S) pi[i] = x[i];
  return true;
}
```

Data structures (3)

StdContainers.cpp

Description: set, multiset and map are sorted, with everything $O(\log n)$. Use their own s.lower_bound(x), as lower_bound(all(s), x) is $O(n)$ on them. unordered_set/map are $O(1)$ on average, but slow on anti-hash tests. pair and tuple compare lexicographically.

dcb762, 34 lines

```
vector<T> v(n, val);
v.push_back(x); v.pop_back(); v.back(); v.resize(n);
v.insert(v.begin() + i, x); v.erase(v.begin() + i); // O(n)
sort(all(v)); v.erase(unique(all(v)), v.end());     // dedupe

set<T> s; multiset<T> ms;
s.insert(x); s.erase(x); s.count(x); s.contains(x);
*s.begin(); *s.rbegin();                  // min, max
s.lower_bound(x); s.upper_bound(x);
ms.erase(x);                              // erases ALL copies of x
ms.erase(ms.find(x));                     // one copy, x must exist

map<K, V> m;
m[k]++;                                   // inserts V() if missing
m.count(k); m.erase(k); m.lower_bound(k);
for (auto& [key, val] : m) {}             // in key order
unordered_set<T> us; unordered_map<K, V> um;

priority_queue<T> pq;                     // max-heap
pq.push(x); pq.top(); pq.pop(); pq.size();
priority_queue<T, vector<T>, greater<T>> minpq;
deque<T> dq;                              // dq[i] O(1)
dq.push_front(x); dq.pop_front(); dq.push_back(x); dq.front();
queue<T> q; q.push(x); q.front(); q.pop();
stack<T> st; st.push(x); st.top(); st.pop();

bitset<N> b;                              // operations O(N / 64)
b[i] = 1; b.count(); b.set(); b.reset(); b.flip(); b <<= k;
b._Find_first(); b._Find_next(i);         // next set bit, N if none

string str;
str.substr(pos, len); str.find(t) == string::npos;
to_string(x); stoi(str); stoll(str);
auto [fst, snd] = pr;
```

BinarySearch.cpp

Description: Binary search on a sorted vector v. lower_bound and upper_bound return v.end() if there is no such element (then the index is v.size()), so check before using *it. x occurs $j - i$ times, the last element $< x$ is at $i - 1$ and the last $\leq x$ at $j - 1$. partition_point returns the first element for which the condition is false; the condition must be true on a prefix of v and false after it. For a vector sorted in descending order, pass greater().
Time: $\mathcal{O}(\log n)$

5ede26, 10 lines

```
sort(all(v));
auto it = lower_bound(all(v), x);  // first >= x
auto it2 = upper_bound(all(v), x); // first > x
if (it != v.end() && *it == x) {}  // x is at it - v.begin()
if (it2 != v.end()) {}             // smallest > x is *it2
auto i = it - v.begin(), j = it2 - v.begin();
bool found = binary_search(all(v), x);
auto k = partition_point(all(vp),  // first p.first >= x
  [&](auto& p) { return p.first < x; }) - vp.begin();
auto i2 = lower_bound(all(desc), x, greater()) - desc.begin();
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.
Time: $\mathcal{O}(\log N)$

0f4bdb, 19 lines

```
struct Tree {
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative fn)
  vector<T> s; int n;
  Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
```

```
    s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

UnionFind.h

Description: Disjoint-set data structure. Keeps track of groups (e.g. connected components) as they are merged. join(a, b) merges the groups of a and b, and returns false if they were already the same group. find(x) returns the representative of x's group, size(x) the group's size. Internally, e[x] is the parent of x, or minus the group's size if x is the representative.
Usage: `UF uf(n); uf.join(a, b); uf.sameSet(a, b);`
Time: $\mathcal{O}(\alpha(N))$, which is basically constant

7aa27c, 14 lines

```
struct UF {
  vi e;
  UF(int n) : e(n, -1) {}
  bool sameSet(int a, int b) { return find(a) == find(b); }
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

de4ad0, 21 lines

```
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

Matrix.h

Description: Basic operations on square matrices.
Usage: `Matrix<int, 3> A;`
`A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};`
`array<int, 3> vec = {1,2,3};`
`vec = (A^N) * vec;`

4da5a2, 26 lines

```
template<class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
    M a;
    rep(i,0,N) rep(j,0,N)
      rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
      if (p&1) a = a*b;
      b = b*b;
      p >>= 1;
    }
    return a;
  }
};
```

Numerical (4)

4.1 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2 m)$

44c9ab, 38 lines

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);

  rep(i,0,n) {
    double v, bv = 0;
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n).
Time: $\mathcal{O}(n^3)$

ebfff6, 35 lines

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }
```

```
  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

Number theory (5)

5.1 Primality

Eratosthenes.cpp

Description: Sieve of Eratosthenes: finds all primes up to MAXP. Call sieve() once; then isPrime[x] says whether $x \leq$ MAXP is prime, and primes lists the primes in increasing order. Each prime crosses out its multiples, starting at i^2 as smaller multiples have a smaller prime factor. There are 664 579 primes below 10^7.
Time: $\mathcal{O}(N \log \log N)$

bfa652, 11 lines

```
const int MAXP = 1e7;
vector<bool> isPrime(MAXP + 1, true);
vi primes;
void sieve() {
  isPrime[0] = isPrime[1] = false;
  rep(i,2,MAXP+1) if (isPrime[i]) {
    primes.push_back(i);
    for (ll j = (ll)i * i; j <= MAXP; j += i)
      isPrime[j] = false;
  }
}
```

5.2 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Maps a permutation of 0..n-1 to a unique integer in [0, n!), e.g. to index visited states in a BFS over permutations. Fits in an int for n <= 12. Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

044568, 6 lines

```
int permToInt(vi& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
    use |= 1 << x;                    // (note: minus, not ~!)
  return r;
}
```

Graph (7)

7.1 Fundamentals

GraphBasics.cpp

Description: Graph as adjacency lists g. bfs(s) returns the unweighted shortest distances from s, -1 if unreachable. dfs(u) marks everything reachable from u in vis. Its recursion can get n deep, and 8 MB of stack fits only about $4 \cdot 10^4$ levels (Codeforces gives 256 MB); bfs also works for reachability. Usage counts connected components.
Usage: `int c = 0; rep(u,0,n) if (!vis[u]) c++, dfs(u);`
Time: $\mathcal{O}(n+m)$

46bed4, 27 lines

```
const int MAXN = 2e5 + 5;
int n, m; // nodes 0..n-1, m edges
vector<int> g[MAXN];
void readGraph() {
  cin >> n >> m;
  rep(i,0,m) {
    int a, b; cin >> a >> b; a--, b--; // if 1-indexed
    g[a].push_back(b);
    g[b].push_back(a); // remove if directed
  }
}
vi bfs(int s) {
  vi dist(n, -1);
  queue<int> q;
  q.push(s), dist[s] = 0;
  while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : g[u]) if (dist[v] < 0)
      dist[v] = dist[u] + 1, q.push(v);
  }
  return dist;
}
bool vis[MAXN];
void dfs(int u) {
  vis[u] = true;
  for (int v : g[u]) if (!vis[v]) dfs(v);
}
```

Dijkstra.cpp

Description: Shortest distances from s when no edge weight is negative. adj[u] holds a pair (v, w) for each edge from u to v with weight w (add both directions if undirected). Unreachable nodes get LLONG_MAX. The priority queue always gives the closest node that is not done yet. If you only need s to t, pass t to stop early: then only dist[t] and path(t) are final. par[v] is the node before v on a shortest path, and path(t) follows it back to s.
Usage: `adj[a].push_back({b, w});`
`dijkstra(s, t); ll d = dist[t]; vi p = path(t);`
Time: $\mathcal{O}((n+m)\log m)$

"GraphBasics.cpp" 685eda, 26 lines

```
typedef pair<ll, int> pli;
vector<pair<int, ll>> adj[MAXN];
vector<ll> dist;
vi par;
void dijkstra(int s, int t = -1) {
  dist.assign(n, LLONG_MAX), par.assign(n, -1);
  priority_queue<pli, vector<pli>, greater<pli>> pq;
  dist[s] = 0;
  pq.push({0, s});
  while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue; // u was already done
    if (u == t) break;         // dist[t] is final
    for (auto [v, w] : adj[u]) if (d + w < dist[v]) {
      dist[v] = d + w, par[v] = u;
      pq.push({dist[v], v});
    }
  }
}
vi path(int t) { // s, ..., t, or empty if unreachable
  if (dist[t] == LLONG_MAX) return {};
  vi p;
  for (int u = t; u != -1; u = par[u]) p.push_back(u);
  reverse(all(p));
  return p;
}
```

TopoSort.h

Description: Topological sorting. Orders the nodes of a directed graph so that every edge goes from left to right, e.g. so that each task comes after the tasks it depends on. gr[u] lists the nodes that u has an edge to. Works by repeatedly taking a node with no incoming edges left. If there is a cycle, the result has fewer than n nodes, so this also detects cycles.
Usage: `vi order = topoSort(vector<vi>(g, g + n));`
Time: $\mathcal{O}(|V| + |E|)$

d678d8, 8 lines

```
vi topoSort(const vector<vi>& gr) {
  vi indeg(sz(gr)), q;
  for (auto& li : gr) for (int x : li) indeg[x]++;
  rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
  rep(j,0,sz(q)) for (int x : gr[q[j]])
    if (--indeg[x] == 0) q.push_back(x);
  return q;
}
```

7.2 Trees

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1. Otherwise returns (total weight, par), where par[i] is the parent of node i and par[r] = -1.
Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
```

```
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

Geometry (8)

8.1 Geometric primitives

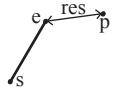
Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

SegmentDistance.h



Description:
Returns the shortest distance between point p and the line segment from point s to e.
Usage: `Point<double> a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10;`

"Point.h" d5f39f, 6 lines

```
typedef Point<double> P;
double segDist(P s, P e, P p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument *eps* is given 0 is returned if p is within distance *eps* from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" ca4665, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(P s, P e, P p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 11 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleTangents.h

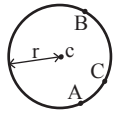
Description: Finds the external tangents of two circles, or internal if r2 is negated. Returns 0, 1 or 2 tangents as (point on circle 1, point on circle 2), with 0 if one circle contains the other (or they overlap, for internal) and 1 if they touch. For the tangents from a point to a circle, use the point as circle 2 with r2 = 0.

"Point.h" b0153d, 13 lines

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

circumcircle.h

Description:



The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.

"Point.h" 6cffc5, 9 lines

```
typedef Point<double> P;
double ccRadius(P A, P B, P C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(P A, P B, P C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$

"circumcircle.h" 09dd0a, 17 lines

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`
Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
```

```
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

PolygonCenter.h

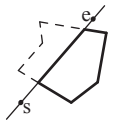
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$

"Point.h" 9706dc, 9 lines

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

PolygonCut.h

Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.



Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`

"Point.h" d07181, 13 lines

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  }
  return res;
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counterclockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). For the farthest pair of a point set, use hullDiameter(convexHull(pts)).
Time: $\mathcal{O}(n)$

"Point.h" c571b8, 12 lines

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. Use e.g. for Euclidean MST (its edges are among the triangulation's edges, so run Kruskal on those) or nearest neighbours. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.
Time: $\mathcal{O}(n \log n)$

"Point.h" eefdf5, 88 lines

```
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}

pair<Q,Q> rec(const vector<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
```

```
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
}
```

8.5 3D

sphericalDistance.h

Description: Returns the shortest (great-circle) distance between two points on a sphere with radius radius. f1, f2 are longitudes (ϕ, from the x axis) and t1, t2 are angles from the north pole (θ, from the z axis), all in radians. For latitude ℓ, use $\theta = \pi/2 - \ell$. For cartesian points on the unit sphere, set dx, dy, dz to their difference.

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
}
```

Strings (9)

Manacher.h

Description: Use for longest palindromic substring (or all palindromes). p[1][i] = k means s[i-k..i+k] is the longest odd palindrome centred at i, p[0][i] = k means s[i-k..i+k-1] is the longest even palindrome centred between i-1 and i. The longest is the max of 2p[1][i]+1 and 2p[0][i].

Time: $\mathcal{O}(N)$

e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).

Time: $\mathcal{O}(\log N)$

edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}

void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

10.2 Misc. algorithms

LIS.h

Description: Compute indices for the longest increasing subsequence.

Time: $\mathcal{O}(N \log N)$

2932a0, 17 lines

```
template<class I> vi lis(const vector<I>& S) {
  if (S.empty()) return {};
  vi prev(sz(S));
  typedef pair<I, int> p;
  vector<p> res;
  rep(i,0,sz(S)) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it = lower_bound(all(res), p{S[i], 0});
    if (it == res.end()) res.emplace_back(), it = res.end()-1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it-1)->second;
  }
  int L = sz(res), cur = res.back().second;
  vi ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.

Time: $\mathcal{O}(N \max(w_i))$

b20ccc, 16 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2*m, -1);
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}
```

10.3 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.

SubsetSums.cpp

Description: Sums over subsets: afterwards D[m] is the sum of the old D[s] over all submasks s of m (masks using only bits of m). The example counts, for each mask m, how many numbers in a fit inside m, e.g. D[5] = 2 as 1 and 4 fit in 101. For sums over supersets, use `if (!(i & 1 << b)) D[i] += D[i | 1 << b]` instead. The same loops with -= undo it, giving back the old D.

Time: $\mathcal{O}\left(K \cdot 2^K\right)$

99d1ee, 7 lines

```
const int K = 3;
vector<int> a = {1, 3, 3, 4};
vector<ll> D(1 << K, 0);
for (int x : a) D[x]++;      // D = [0, 1, 0, 2, 1, 0, 0, 0]
rep(b,0,K) rep(i,0,(1 << K))
  if (i & 1 << b) D[i] += D[i^(1 << b)];
// now D = [0, 1, 0, 3, 1, 2, 1, 4]
```